

DATA266 Lab 1

Team 5

Sneha Singh

Ritika Mukesh Neema

Abstract

This report is Team 5's write-up for DATA266 Lab 1. Sneha Singh and Ritika Mukesh Neema each trained a character-level GPT on TinyStories, three Yelp polarity classifiers, and a CycleGAN for photo-to-Monet transfer. Each person used their own architecture and their own hyperparameters. Each part below gives one person's run, then the other person's run, then a comparison of the two. Kaggle FID scores are reported per member in Part 3.

The repository is https://github.com/snehas-SJSU/Data_266-Lab1.

Introduction

Each part below has two individual write-ups and then one team comparison. Section A is Sneha's model, metrics, failures, and hardware. Section B is Ritika's. Section C puts the two runs side by side. Code for Sneha lives under `sneha_singh/`. Code for Ritika lives under `ritika_mukesh_neema/`. Raw TinyStories and the Monet and photo folders are linked from the README because they are too large for git. Yelp is downloaded by the notebook from Hugging Face.

What each of us built

Part	Sneha Singh (her own model)	Ritika Mukesh Neema (her own model)
1 — GPT (TinyStories)	4 layers, emb 256, context 128, 10 epochs, RTX 5090	4 layers, emb 128, context 256, 12 epochs, Colab Tesla T4
2 — Yelp polarity	BiLSTM best, macro-F1 0.942, full train set, Colab Tesla T4	BiLSTM best, macro-F1 0.934, full 560k set, Colab Tesla T4
3 — CycleGAN	Batch 4, 100 epochs, Colab NVIDIA L4, about 5.4 h, photo→Monet FID 98.29	Batch 4, 125 epochs, Colab NVIDIA A100, about 4.5 h, photo→Monet FID 93.89

We share the raw data. The rows above are two separate runs of the same part, not a split of who had to do which part.

Part 1 — GPT from scratch

1.A Sneha Singh — individual

I trained a character-level GPT on TinyStories. The attention code is mine. I did not use `nn.Transformer` or `nn.MultiheadAttention`.

The tokenizer is a simple `char_to_idx / idx_to_char` map, not BPE. The model has 4 layers, 4 heads, embedding size 256, and a context of 128 characters. A causal mask stops each position from seeing the future. I tied the token weights the usual way and trained with AdamW. Bias and LayerNorm do not get weight decay.

The reported run is the full one, not the old 2-epoch smoke test. I took my own split from `TinyStories-train.txt`: 100,000 train windows and 10,000 validation windows, seed 670. That text file is 1.79 GB, over GitHub's 100 MB limit, so it is not in git:

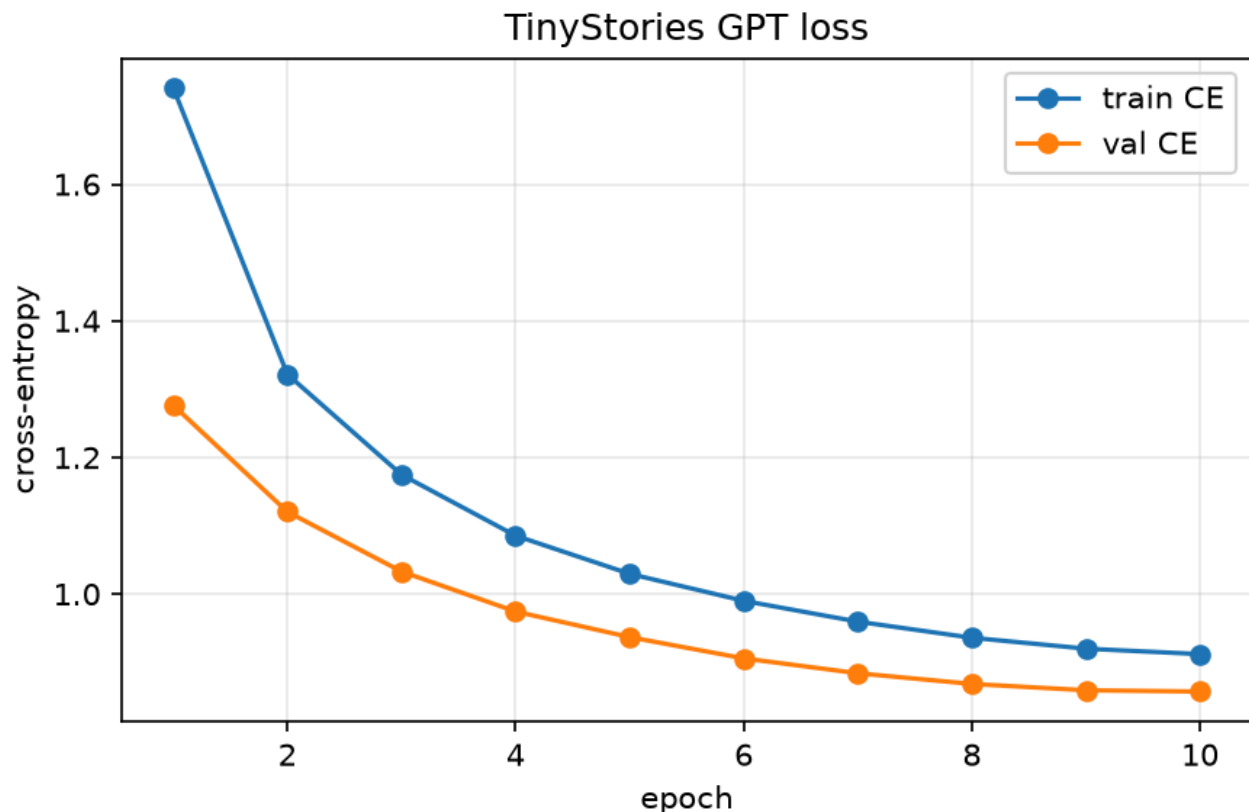
https://drive.google.com/drive/folders/12MFVOo6T3QRW3X6THiDkh5svtNgL13W_?usp=share_link

I trained for 10 epochs on an NVIDIA GeForce RTX 5090 with mixed precision. Learning rate was 0.001, with 100 warmup steps and then cosine decay. Batch size was 32. The run took 457 seconds and peaked at about 349 MB. There were no NaN losses. The checkpoint `best.pt` is 12 MB, so it is on git.

Checkpoint id: `task1_llm/sneha_singh/checkpoints/best.pt`. Log: `reproducibility/raw_logs/sneha_singh/task1_llm/train_full.log`.

Metric	Value
Train cross-entropy	0.912
Val cross-entropy	0.857
Perplexity	2.36
Bits per character	1.236
Generalization gap (val – train)	–0.055
Top-1 next-character accuracy	0.729
Distinct-1 / 2 / 3	0.225 / 0.655 / 0.856
Repeated 4-gram rate	0.120
Max gradient norm / NaN count	5.42 / 0
Parameters	3.25M
Train tokens/sec	280,325
Generation tokens/sec	409
Peak memory / train time	349 MB / 457 s

The result I care about on this model is that the samples stay varied while the loss stays stable. Distinct-1 is 0.225, distinct-2 is 0.655, and distinct-3 is 0.856. The repeated 4-gram rate is 0.120. Next-character top-1 accuracy is 0.729. Training had no NaNs, peaked at about 349 MB, and ran at about 280,000 tokens per second. The loss falls smoothly and the validation curve stays close to the training curve. That is what I wanted from 10 epochs on this size of model.



Part 1 training and validation loss

The prompt I used for samples was “some changes. She added some nice colors”.

Failure cases

- **Repetition.** Greedy decoding loops: “You are very happy. You are very happy.”
- **Broken word.** The same decode prints “them them”. A character model does not know where a word ends.
- **Story drift.** Temperature 0.8 moves from colors to a box, a door, and a tree. The model only sees 128 characters at a time.

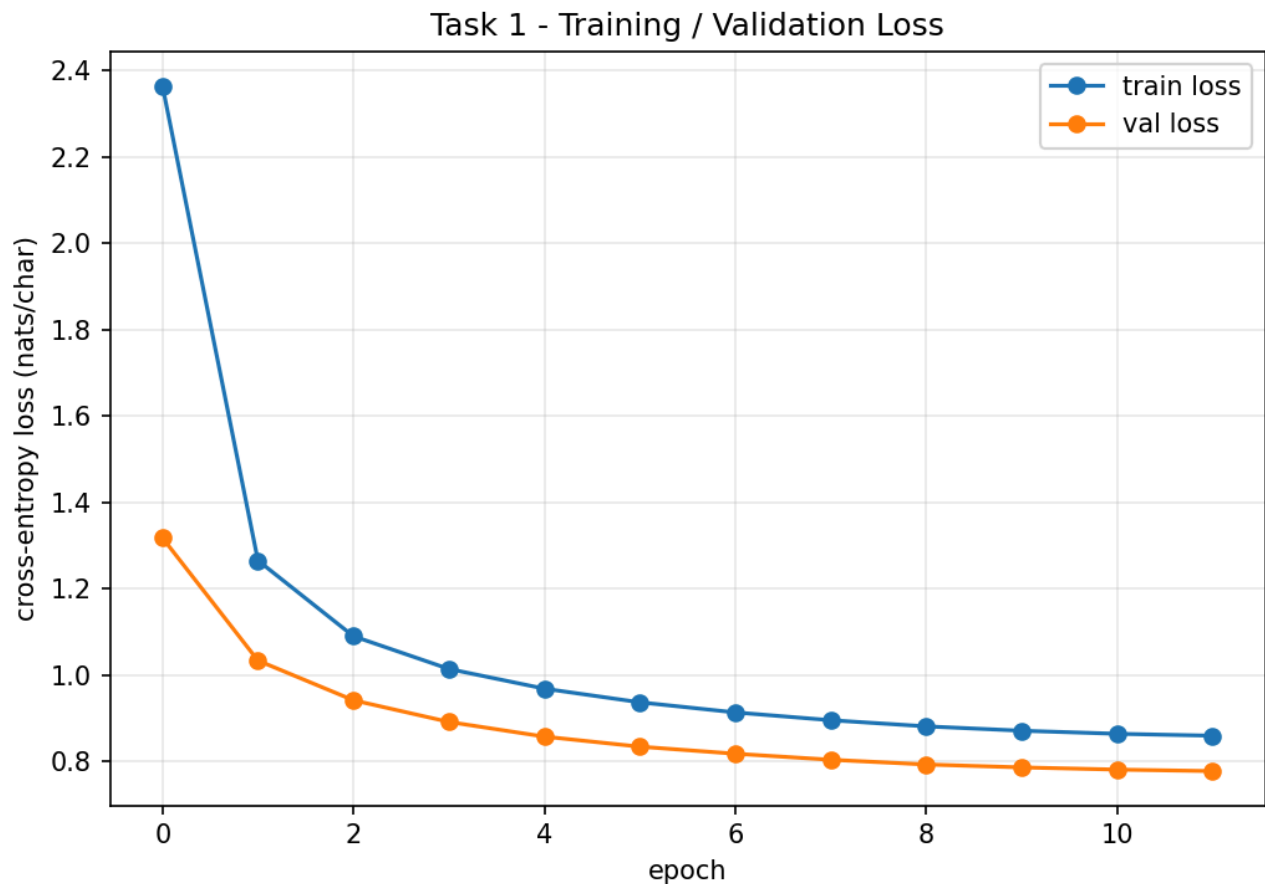
Full cases: [task1_llm/sneha_singh/failure_analysis.md](#)

1.B Ritika Mukesh Neema — individual

I wrote the attention myself. I did not use `nn.Transformer` or `nn.MultiheadAttention`. My model has 4 layers, 4 heads, embedding size 128, and a context of 256 characters. The tokenizer is character-level, built only from my training text, with a vocab of 221. I drew my own 100,000 / 10,000 windows from the full TinyStories train file, seed 6638. The checkpoint is `task1_llm/ritika_mukesh_neema/checkpoints/ckpt_final.pt`.

I trained for 12 epochs, batch 64, learning rate 0.0003, with linear warmup and cosine decay. Weight decay was 0.01 and gradients were clipped at 1.0. The run was on a Colab Tesla T4 and took 1,727 seconds, peaking at 1,749 MB. There were no NaN losses.

Metric	Train	Val
Cross-entropy	0.860	0.778
Perplexity	2.36	2.18
Bits per character	1.240	1.122
Generalization gap		-0.082
Top-1 next-character accuracy		0.755
Distinct-1 / 2 / 3		0.008 / 0.054 / 0.137
Repeated 4-gram rate		0.206
Max gradient norm / NaN count		4.99 / 0
Parameters		0.88M
Train tokens/sec		177,917
Generation tokens/sec		239
Peak memory / train time		1,749 MB / 1,727 s



Part 1 training and validation loss

Failure cases

- **Repetition.** All 10 greedy samples are the same story, and it loops: “You are very happy. You are very happy.”
- **Broken word.** Temperature sampling writes “designt”, which is not English.
- **Story drift.** The object moves from a box to a red ball to a bird.
- **Does not stop.** Generation does not stop at the story-boundary token, so a second story gets stuck on the end.

Full cases: [task1_llm/ritika_mukesh_neema/failure_analysis.md](https://github.com/ritika-mukesh-neema/task1_llm/blob/main/failure_analysis.md)

1.C Team comparison

	Sneha	Ritika
Architecture	4 layers, 4 heads, emb 256, context 128, char tokenizer, causal mask	4 layers, 4 heads, emb 128, context 256, char tokenizer, causal mask
Hyperparameters	10 epochs, batch 32, lr 0.001, warmup 100, seed 670, AdamW	12 epochs, batch 64, lr 0.0003, warmup + cosine, seed 6638, clip 1.0
Train / val CE	0.912 / 0.857	0.860 / 0.778
Perplexity / bits per char	2.36 / 1.236	2.18 / 1.122
Gen. gap / top-1 acc	-0.055 / 0.729	-0.082 / 0.755
Distinct-1/2/3	0.225 / 0.655 / 0.856	0.008 / 0.054 / 0.137
Repeated 4-gram / NaNs	0.120 / 0	0.206 / 0
Max gradient norm	5.42	4.99
Params / time / peak MB	3.25M / 457 s / 349	0.88M / 1,727 s / 1,749
Train / gen tokens per sec	280,325 / 409	177,917 / 239
Checkpoint	best.pt	ckpt_final.pt
Log	train_full.log	train_full.log

Strength. Ritika's validation loss is 0.778, and Sneha's is 0.857. Sneha's distinct-1 is 0.225, and Ritika's is 0.008. Each run leads on a different metric.

Weakness. Both greedy decoders loop on "You are very happy."

Limitation. Embedding size, context length, and the machine differ, so the loss gap is not a pure architecture contest.

Next. Both of us would add a repetition penalty at decode time, and stop generation when the story-boundary token appears.

Part 2 — Yelp polarity

2.A Sneha Singh — individual

This is Yelp polarity, not IMDB. I learned the embeddings from scratch. I did not use a pretrained language model.

The notebook downloads `fancyzhx/yelp_polarity` from Hugging Face when the run starts.

I wanted one simple model and two that can use word order, so a strong baseline would be obvious if the fancier models did not beat it.

1. **Baseline.** Mean pool over the embeddings, then a linear layer.
2. **BiLSTM.** Reads the review in both directions, so negation and longer sentences have a chance.
3. **TextCNN.** Filters of width 3, 4, and 5, meant to catch short phrases like “not good”.

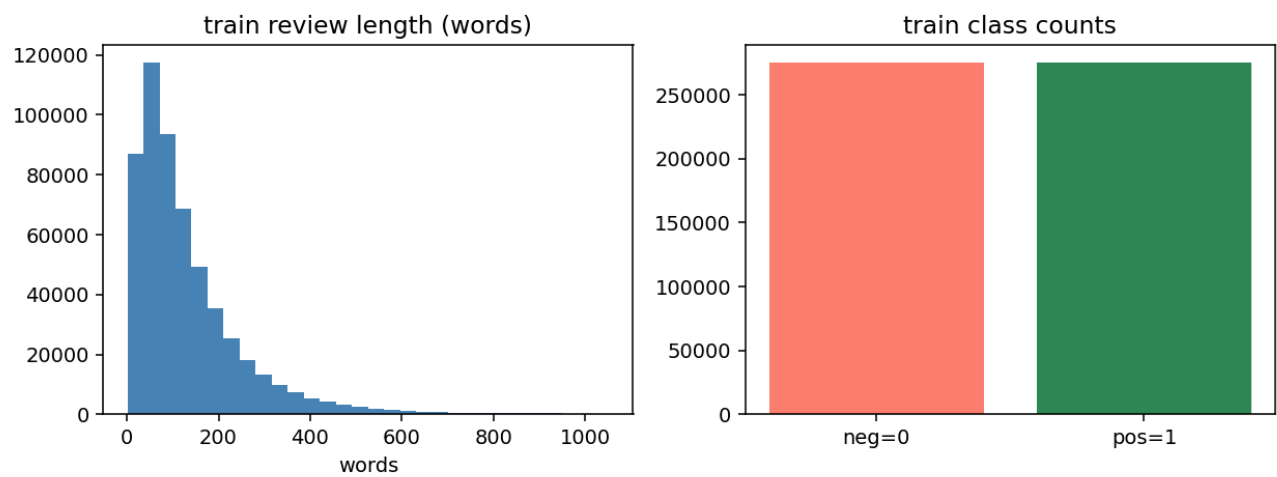
I lowercased, stripped punctuation, removed stopwords, and lemmatized. The vocabulary comes from the training text only. I held out 10,000 validation reviews from the 560,000-row train file, trained on 549,953 reviews, and tested on all 38,000 official test reviews. Five epochs, batch 64, embedding size 100, maximum length 128. All three models ran on a Colab Tesla T4. Peak memory was 228 MB, 344 MB, and 1,656 MB.

Checkpoints: `task2_sentiment/sneha_singh/checkpoints/baseline_full.pt`,
`experimental_a_full.pt`, and `experimental_b_full.pt`. Logs:
`reproducibility/raw_logs/sneha_singh/task2_sentiment/baseline_full.log`,
`experimental_a_full.log`, and `experimental_b_full.log`.

Model	Acc	Macro-F1	ROC-AUC	MCC	Brier	ECE	Time
Baseline	0.932	0.932	0.979	0.863	0.052	0.006	233 s
BiLSTM	0.942	0.942	0.986	0.883	0.044	0.012	794 s
TextCNN	0.937	0.937	0.984	0.875	0.047	0.008	3646 s

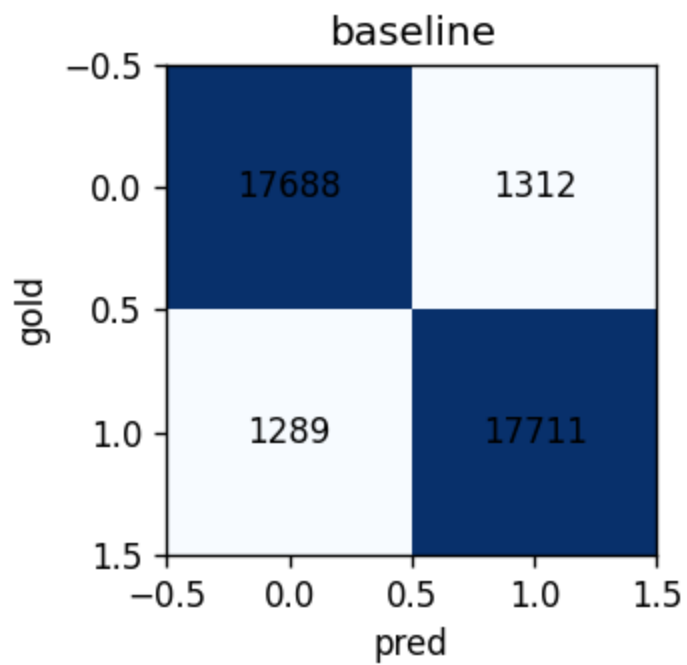
The BiLSTM has the best macro-F1. McNemar against the baseline is $p < 0.001$ for both the BiLSTM and the TextCNN, so on this 38k test set both beat the mean pool. Short reviews are a bit easier than long ones. Long-review macro-F1 is 0.927, 0.934, and 0.933 for baseline, BiLSTM, and TextCNN.

The length and class balance of this run:

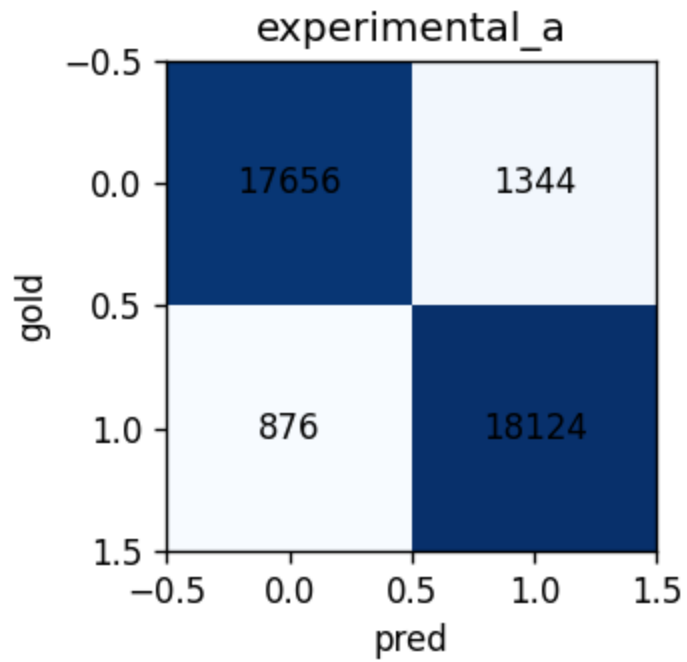


Yelp length and class balance

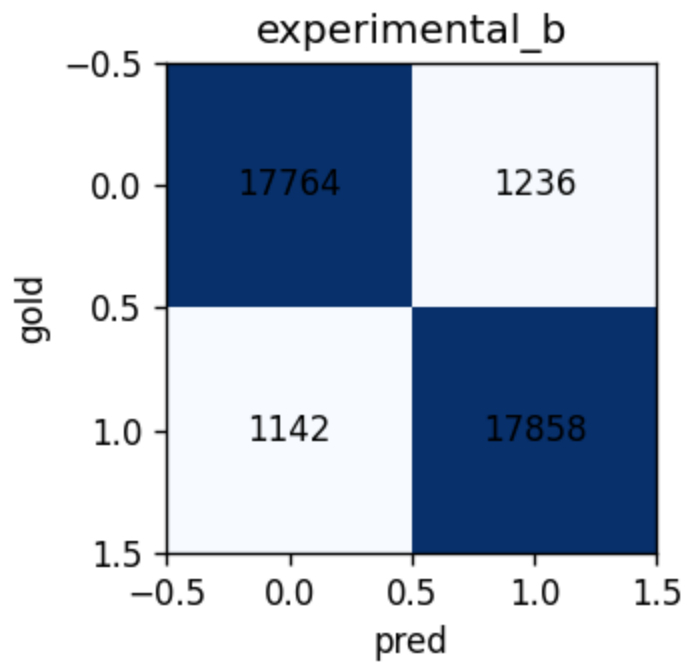
Confusion matrices on the test set:



Baseline confusion matrix



BiLSTM confusion matrix



TextCNN confusion matrix

Error review

I read 20 BiLSTM mistakes: five confident false positives, five confident false negatives, five near the decision threshold, and five long-review misses.

- **Confident false positive.** Gold negative, predicted positive: “wow love place everything clean new”.
- **Confident false negative.** Gold positive, predicted negative: “look know cox suck fact terrible business”.
- **Near the threshold.** “wow seems like taco bell arizona”.
- **Long review.** “given mixed review quite sure expect”.

The ones that show up most are praise words inside a negative review, a complaint that later turns around, and long reviews where the stars and the sentences pull apart. The fix I wrote down for these cases is to balance review length or train longer, and to add negation handling.

Full 20-row table: [task2_sentiment/sneha_singh/failure_analysis.md](https://github.com/sneha-singh/task2_sentiment/blob/master/failure_analysis.md)

2.B Ritika Mukesh Neema — individual

I trained the same three families, with embeddings learned from scratch. My baseline is a mean pool plus a linear layer. My BiLSTM is there so negation can depend on word order. My TextCNN uses filter widths 3, 4, and 5 with global max-pool. I lowercased, stripped punctuation and HTML, removed stopwords, and used Porter stemming instead of lemmatization. I trained on the full Yelp polarity set, about 560,000 reviews, on a Colab Tesla T4. Checkpoints are `task2_sentiment/ritika_mukesh_neema/checkpoints/baseline.pt`, `task2_sentiment/ritika_mukesh_neema/checkpoints/bilstm.pt`, and `task2_sentiment/ritika_mukesh_neema/checkpoints/textcnn.pt`. I do not have confusion-matrix images in the repo. The numbers below are from my `metrics_report.csv`.

Model	Acc	Macro-F1	ROC-AUC	PR-AUC	MCC	Brier	ECE	Time	Peak MB
Baseline	0.922	0.922	0.974	0.973	0.843	0.059	0.007	125 s	2,123
BiLSTM	0.934	0.934	0.982	0.982	0.868	0.051	0.025	798 s	2,274
TextCNN	0.926	0.926	0.979	0.979	0.852	0.055	0.021	267 s	2,450

My BiLSTM is the best of my three. McNemar against my baseline is $p < 0.001$ for the BiLSTM and $p = 0.00022$ for the TextCNN, so on my test set both sequence models beat the mean pool.

Error review

I reviewed 20 TextCNN mistakes.

- **Confident false positive.** Sarcasm, with no negative words: “Is there REALLY even a Leonard...” A French review in the same set also fails, because the stemmer is English-only.
- **Confident false negative.** A positive review opens with “Cox sucks” before it turns around.
- **Near the threshold.** “The employees at this Target seemed unusually friendly...”
- **Long review.** “I used to love D&B... it has gone down hill.”

The fixes I wrote down are a language filter for non-English reviews, more weight on the words after “however”, and treating a score near 0.5 as a calibration margin rather than a hard error.

Full 20-case write-up: [task2_sentiment/ritika_mukesh_neema/failure_analysis.md](#)

2.C Team comparison

Same three model families. The tables use the same columns. Both of us trained on the full Yelp train file. Sneha tested on the official 38,000 reviews. Ritika tested on her own split of that file.

Sneha Singh. 549,953 / 10,000 / 38,000, lemmatize, max length 128, Colab Tesla T4. Checkpoints: `baseline_full.pt`, `experimental_a_full.pt`, `experimental_b_full.pt`.

Model	Acc	Macro-F1	ROC-AUC	PR-AUC	MCC	Brier	ECE	McNemar p	Params	Time	Peak MB
Baseline	0.932	0.932	0.979	0.979	0.863	0.052	0.006	—	10.68M	233 s	228
BiLSTM	0.942	0.942	0.986	0.986	0.883	0.044	0.012	< 0.001	10.76M	794 s	344
TextCNN	0.937	0.937	0.984	0.985	0.875	0.047	0.008	< 0.001	10.75M	3646 s	1,656

Ritika Mukesh Neema. Full 560k set, Porter stem, max length 200, Colab Tesla T4. Checkpoints: `baseline.pt`, `bilstm.pt`, `textcnn.pt`.

Model	Acc	Macro-F1	ROC-AUC	PR-AUC	MCC	Brier	ECE	McNemar p	Params	Time	Peak MB
Baseline	0.922	0.922	0.974	0.973	0.843	0.059	0.007	—	3.00M	125 s	2,123
BiLSTM	0.934	0.934	0.982	0.982	0.868	0.051	0.025	< 0.001	3.24M	798 s	2,274
TextCNN	0.926	0.926	0.979	0.979	0.852	0.055	0.021	0.00022	3.12M	267 s	2,450

Precision, recall, the other F1 scores, bootstrap intervals, examples per second, and slice scores are in the next table. Slice scores are filled where the metrics file has a short-review or long-review column.

Metric	Sneha baseline	Sneha BiLSTM	Sneha TextCNN	Ritika baseline	Ritika BiLSTM	Ritika TextCNN
Precision macro	0.932	0.942	0.937	0.922	0.934	0.926
Recall macro	0.932	0.942	0.937	0.922	0.934	0.926
F1 micro	0.932	0.942	0.937	0.922	0.934	0.926
F1 weighted	0.932	0.942	0.937	0.922	0.934	0.926
Acc 95% CI	0.929–0.934	0.939–0.944	0.935–0.940	0.919–0.925	0.931–0.936	0.924–0.928
Macro-F1 95% CI	0.929–0.934	0.939–0.944	0.935–0.940	0.919–0.925	0.931–0.936	0.923–0.928
MCC 95% CI	0.859–0.868	0.879–0.888	0.870–0.879	0.838–0.849	0.863–0.873	0.847–0.857
Examples/sec	11,807	3,464	754	20,181	3,159	9,432
Short macro-F1	0.934	0.947	0.940			
Short error rate	0.065	0.052	0.059			
Long macro-F1	0.927	0.934	0.933			
Long error rate	0.072	0.065	0.066			

Strength. Sneha’s BiLSTM macro-F1 is 0.942. Ritika’s BiLSTM macro-F1 is 0.934. On both runs the BiLSTM beats that person’s baseline, and McNemar says the lead is real.

Weakness. Both of us miss sarcasm, mixed reviews, and negation.

Limitation. The test splits are not the same cut of the file. Sneha lemmatizes and uses a maximum length of 128. Ritika stems and uses a maximum length of 200. The scores are not one shared test.

Next. Both of us would use the same maximum review length, so long reviews are scored the same way.

Part 3 — CycleGAN, photo to Monet

3.A Sneha Singh — individual

I trained an unpaired CycleGAN. The Kaggle direction is photo to Monet. The generators are mine. I did not use a pretrained model to make the images. Inception and the other nets in the eval script are only for measuring.

Each generator is a ResNet with 9 blocks at 256×256 , 64 base filters, reflection padding, instance norm, and a tanh output. Upsampling is nearest-neighbor times two, then a stride-1 convolution, not a transposed convolution. Each discriminator is a PatchGAN trained with least-squares loss, and every image it sees, real or fake, goes through DiffAugment (colour, translation, cutout). The losses are adversarial, cycle L1 with $\lambda = 10$, and identity with weight 5. The generators also keep an EMA copy (decay 0.999). Real labels are smoothed to 0.9. An image pool of 50 feeds the discriminator. Batch size is 4. I trained 50 epochs at a constant learning rate and 50 more with linear decay. Each epoch is 800 steps and resamples 3,200 photos. The shorter Monet loader is cycled so those photos actually get used. pred_B2A is photo→Monet (7,038 images). pred_A2B is Monet→photo (300 images). Those are the professor folder names.

Epochs 1–84 ran on a Colab NVIDIA A100 and epochs 85–100 were resumed on an NVIDIA RTX 4060 Laptop GPU, with mixed precision. It took 19,836 s (about 5.5 hours) and peaked near 6,855 MB, with 28.3M parameters. I scored epochs 76–100, raw and EMA weights, with the professor method (checkpoint_scores_epochs76_100.csv); epoch 91 (raw weights) scored best and is checkpoints/best.pt (87 MB, generators only, in git). For Monet→photo the output is averaged over 6 views of the input (original, horizontal flip, four 4-pixel shifts); photo→Monet is a single pass. make_submission.py rebuilds both folders from best.pt.

The Monet and photo folders are in one Drive folder:

https://drive.google.com/drive/folders/1BXYfhW8uZ6umK1TZFW8Un62mZVK72L7Y?usp=share_link

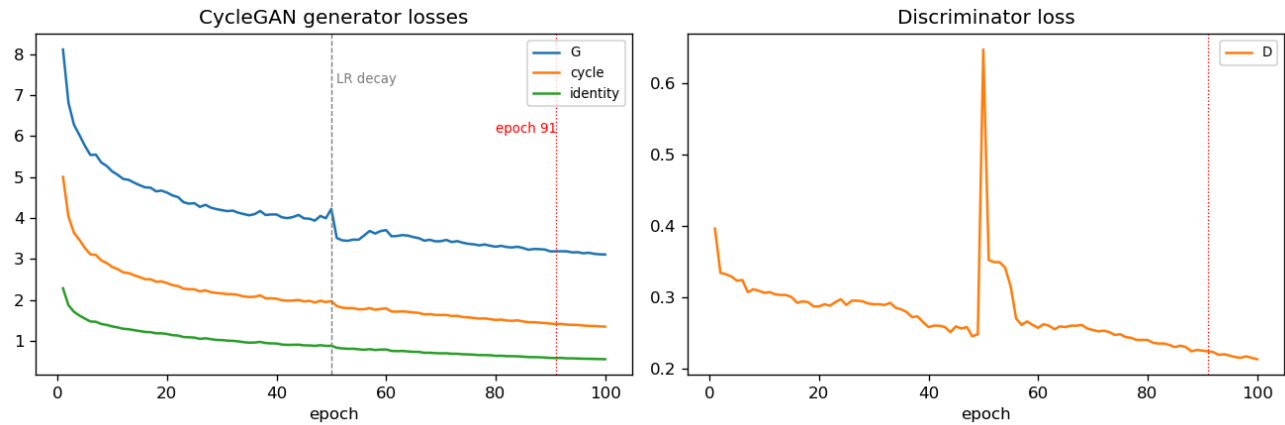
Direction	FID	MiFID
Photo → Monet (pred_B2A)	95.78	0.406
Monet → photo (pred_A2B)	94.83	0.412

FID is the Fréchet distance between Inception features of the generated images and the real target images. MiFID is the average cosine distance of those features on the first 300 sorted images, which is the professor script. Lower is better on both. FID and MiFID were computed on GPU with the unchanged professor script. submission.csv is the average of both directions: FID 95.30, MiFID 0.409, score 47.857.

Epochs 85–100 scored between 47.8 and 49.4, so the model had plateaued. View averaging lowered Monet→photo FID from 96.5 to 94.8; for photo→Monet it did not help, so only Monet→photo uses

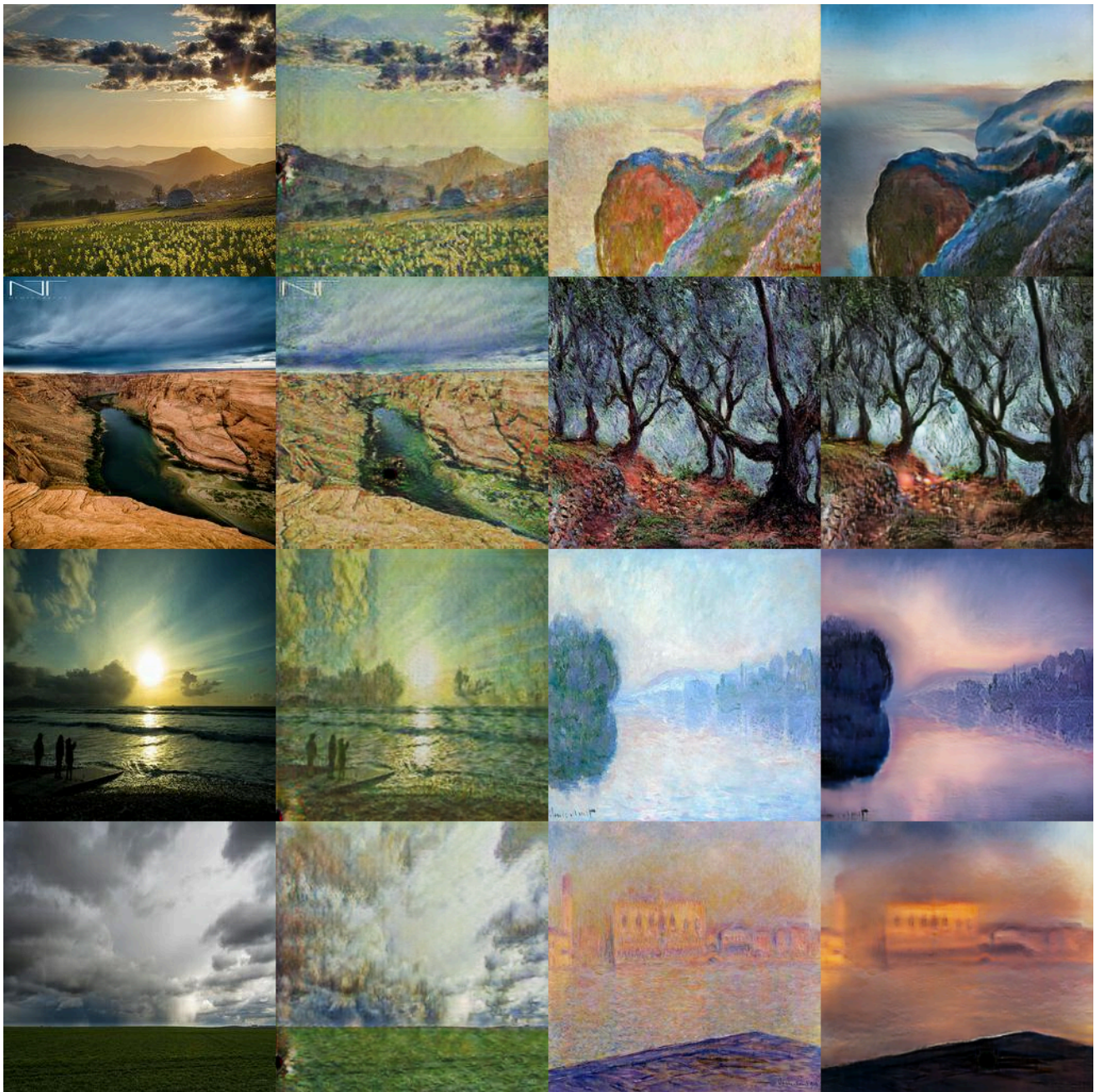
it. My earlier 32-filter run without EMA scored 50.13.

Losses fell across the 100 epochs. The discriminator loss stayed near 0.3 for the first 40 epochs, jumped to 0.65 at epoch 50, and fell to about 0.22 by epoch 91. The generator is not winning that fight, and I think that is why the brush texture gets noisy.



CycleGAN losses

The grid is four photos, the Monet version of each, four real Monets, and the photo version of those.



Photo, generated Monet, real Monet, generated photo

Human audit

What I saw in my audit of an earlier run of the same CycleGAN, and in this grid:

- **Grain.** Skies and water pick up a repeated dab texture.
- **Color wash.** Some skies go flat, and sunsets brown out at the edges.
- **Soft reverse.** A few Monet-to-photo frames go soft or pick up a dark blob.
- **Content.** The layout of the photo usually survives. I did not mark any of my 30 as a copied training Monet.

On the 0–2 sheet my averages were style 1.40 and content 1.67, where higher is better, and artifacts 1.43, where higher is worse. The audit was rated on images from an earlier checkpoint of this CycleGAN (80-epoch run, same 30 indices) and was not repeated for the final epoch-91 checkpoint.

Full notes: [task3_gan/sneha_singh/failure_analysis.md](https://github.com/sneha-singh/failure_analysis.md)

Audit sheet: [task3_gan/sneha_singh/outputs/human_audit/audit_30.csv](https://github.com/sneha-singh/outputs/human_audit/audit_30.csv)

3.B Ritika Mukesh Neema — individual

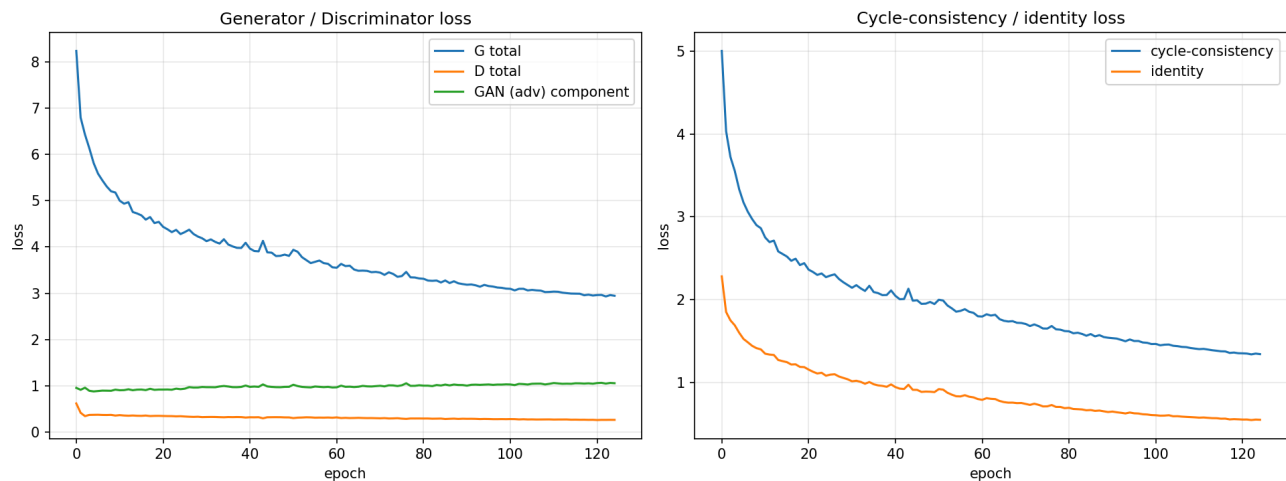
I trained my own CycleGAN: two ResNet-9 generators with 64 base filters, instance norm, reflection padding, and nearest-neighbour upsampling followed by a 3×3 convolution, plus two 70×70 PatchGAN discriminators. The losses are least-squares adversarial, cycle L1 with $\lambda = 10$, and identity L1 with weight 5, with an image pool of 50. Every image the discriminators see goes through DiffAugment (color, translation, cutout), real labels are 0.9, and I keep an EMA copy (decay 0.999) of both generators. Batch size was 4, with 800 steps per epoch. I trained 50 epochs at a constant learning rate of 2e-4 and 75 with linear decay, seed 6638, with mixed precision on a Colab NVIDIA A100. The run took 16,310 seconds (4.5 h), 100,000 steps, and there were no NaN losses. All settings are in `src/config.json`.

This is my second run. The first one trained for 80 epochs of only 300 images each, 24,000 images per domain in total, with transposed-convolution upsampling and no augmentation. Its photo-to-Monet FID was 123.70 on my local metric. The new run sees 400,000 images per domain. The old files are in `outputs/history_v1/`.

I scored the last 30 epochs with the TA's evaluation notebook, raw and EMA weights, 60 candidates in `outputs/checkpoint_scores.csv`. The best is epoch 110 with raw weights, `checkpoints/best.pt`. The checkpoints are over GitHub's 100 MB limit, so they are on Google Drive (link in `checkpoints/README.md`). From that checkpoint, `generate.py` writes all 7,038 photo→Monet and 300 Monet→photo images, and the TA's script (`evaluate_local.py`) gives FID 95.6037 and MiFID 0.4054, a score of 48.0046 (`submission.csv`).

Direction	FID (TA)	MiFID (TA)	KID	Precision	Recall	Cycle L1	LPIPS	Content cosine
Photo → Monet (my B2A)	93.892	0.3991	0.0046	0.477	0.637	0.069	0.142	0.829
Monet → photo (my A2B)	97.316	0.4118	0.0150	0.713	0.383	0.064	0.194	0.730

KID, precision/recall, cycle L1, LPIPS (input vs reconstruction) and content cosine come from `src/full_metrics.py`. Final generator loss was 2.95, discriminator loss 0.26, cycle loss 1.34, identity loss 0.55. Mean gradient norm was 32.2, max 469.9, NaN count 0. 28.3 million parameters, 24.5 images/sec, peak GPU memory 17,368 MB. Monet→photo outputs average 6 flipped and shifted views; photo→Monet is a single pass, because averaging blurred the brush strokes.



CycleGAN losses

Failure notes

- **Flat areas.** Smooth skies and gradients turn into streaky brush texture, worst on a beach-at-dusk photo.
- **Night scenes.** Dark photos drift toward Monet's daylight palette, and a moon becomes an orange smear.
- **Monet → photo.** Outputs keep the layout but stay painterly and dark. Recall is 0.383, so they cover a narrow part of the photo domain.
- **Noise and selection.** Late epochs move by about ± 0.5 in score, and the best epoch is chosen on the images the TA script scores.
- **Human audit.** Two independent raters scored 30 blinded photo→Monet samples from 1 to 5. Means: style 3.35, content 4.22, artifacts 3.05, overall 3.54. Cohen's kappa 0.17 / 0.20 / 0.59 (exact agreement 50% / 47% / 73%); the raters are within one point on at least 96.7% of images. Both name the same problems: streaky flat skies, dark corner blobs, watermarks. Full write-up: [task3_gan/ritika_mukesh_neema/failure_analysis.md](#).

3.C Team comparison

Both models are ResNet-9 CycleGANs with a PatchGAN and least-squares loss. Lower FID is better. Checkpoint links are in sections 3.A and 3.B.

	Sneha photo → Monet	Sneha Monet → photo	Ritika photo → Monet	Ritika Monet → photo
Epochs / batch	50 + 50, batch 4	same run	50 + 75, batch 4	same run
FID	95.78	94.83	93.89	97.32
KID	–	–	0.005	0.015
Precision	–	–	0.477	0.713
Recall	–	–	0.637	0.383
Cycle L1	–	–	0.069	0.064
LPIPS	–	–	0.142	0.194
Content cosine	–	–	0.829	0.730
Params	28.3M	same run	28.3M	same run
Time / images per sec	19,836 s / 16.1	same run	16,310 s / 24.5	same run
Peak memory	6,855 MB	same run	17,368 MB	same run
NaN count	0	same run	0	same run
Generator loss	3.191	same run	2.946	same run
Discriminator loss	0.224	same run	0.263	same run
Cycle loss	1.420	same run	1.342	same run
Identity loss (weighted)	0.589	same run	0.549	same run
Gradient norm	–	same run	32.18 mean, 469.9 max	same run

Human audit score on the 30 photo → Monet images. Style and content are 0–2, higher is better. Artifacts are 0–2, higher is worse. Cohen’s kappa is the inter-rater agreement for both of us on the same 30 images.

	Sneha	Ritika	Cohen’s kappa
Style	1.40		
Content	1.67		
Artifacts	1.43		

Human audit of Ritika’s model (task3_gan/ritika_mukesh_neema/outputs/human_audit/): 30 blinded photo → Monet samples from best.pt, scored by two independent raters on a 1–5 scale (5 = best; for artifacts, 5 = no visible artifacts).

	Rater A	Rater B	Mean	Exact agreement	Cohen's kappa
Style	3.47	3.23	3.35	50.0%	0.17
Content	4.47	3.97	4.22	46.7%	0.20
Artifacts	3.13	2.97	3.05	73.3%	0.59
Overall			3.54		

Strength. Sneha's photo-to-Monet FID is 95.78. Ritika's is 93.89. With the TA script, Sneha's `submission.csv` scores 47.86 and Ritika's 48.00.

Weakness. Sneha's discriminator loss ended at 0.22, Ritika's at 0.26. On Sneha's images the brush texture is grainy; Ritika's flat skies turn streaky.

Limitation. Schedule, hardware and training time differ, and the tools behind KID, precision/recall and LPIPS differ between our two metric scripts, so only the TA-script FID/MiFID numbers are directly comparable.

Next. Sneha's epochs 85–100 scored within two points of each other, so a lower score needs a different training setup rather than more epochs. The class upload is Sneha's epoch 91: `submission.csv` with the average FID and MiFID of both directions.

Final comparison

Part	Sneha Singh	Ritika Mukesh Neema
1 — validation loss	4 layers, emb 256, context 128. 0.857	4 layers, emb 128, context 256. 0.778
1 — distinct-1	4 layers, emb 256, context 128. 0.225	4 layers, emb 128, context 256. 0.008
2 — best model	BiLSTM (lemmatize), 550k train, 38k test. Macro-F1 0.942	BiLSTM (Porter stem), full set. Macro-F1 0.934
3 — photo → Monet FID	Batch 4, about 5.5 h. 95.78 (submission average FID 95.30)	Batch 4, about 4.5 h. 93.89 (submission average FID 95.60)

Kaggle score

Team PairProgramming_Team_05. Score is (FID + MiFID) / 2, each averaged over both directions, computed with the course evaluation script. Final submission.csv: FID 95.30, MiFID 0.409.

Team	Final submission score	Leaderboard rank
PairProgramming_Team_05	47.857	9

The leaderboard lists each team's best entry. Our rank 9 comes from an earlier upload (−45.1953) made before we switched to the course evaluation script. The final submission, selected for the final score, is 47.857 (an earlier course-script upload scored 50.1253).

Citation

1. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. 2017. <https://arxiv.org/abs/1706.03762>
2. Ronen Eldan and Yuanzhi Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? 2023. <https://arxiv.org/abs/2305.07759>
3. Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. 2017. <https://arxiv.org/abs/1703.10593>